

HQT-TI: An Efficient Hilbert Curve Based Index for Spatial Keyword Queries

Lianyin Jia , Yongwang Miao , Suprio Ray , Jiaman Ding , Xiaodong Fu , and Xiuxing Li 

Abstract—This paper introduces HQT-TI, a novel indexing method designed to improve the efficiency of spatial keyword queries. HQT-TI consists of two main components: a Hilbert-QuadTree (HQT) based spatial index and a Trie-Inverted index (TI) combined textual index. HQT integrates the Hilbert curve with a Quadtree, establishing a direct relationship between the two. TI combines a trie and inverted index to minimize the intersection cost associated with long lists, thus improving the speed of keyword queries. The HQT based Spatial Query algorithm (HQT-SQ) reduces overlap checks and limits irrelevant object retrieval by employing query drill-down and depth-first search with limited breadth expansion in spatial queries. Meanwhile, the Segment List Intersection based Keyword Query algorithm (SLI-KQ), built on TI, efficiently handles segment list intersections for keyword queries. The combination of HQT-SQ and SLI-KQ results in HS-SK, a highly efficient spatial keyword query algorithm. Extensive experimental results demonstrate that HS-SKQ outperforms SFC-Quad by up to two orders of magnitude, achieving up to a $5.46\times$ speedup over the best existing competitors, making it a promising solution for large-scale spatial keyword query processing.

Index Terms—Spatial keyword query, Hilbert curve, query drill-down, HQT, HS-SKQ.

I. INTRODUCTION

WITH the advances in mobile internet and wireless communication technologies, a large amount of spatial-textual data containing both location and textual attributes have emerged. For example, social networking applications like Twitter and Facebook allow users to send messages with location information. More than 20% of Google searches and 53% of Bing searches have location intent [1]. Efficient management and querying of large spatial-textual data have become one of the hot research topics in recent years.

Received 1 May 2025; revised 14 April 2026; accepted 28 May 2026. Date of publication 2 June 2026; date of current version 8 August 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 62262035 and Grant 62262034, in part by Science and Technology Projects of Yunnan Universities Serving Key Industries under Grant FWCY-QYCT2025011, in part by Yunnan Provincial Major Science and Technology Special Plan Projects under Grant 202502AD080011, and in part by China Scholarship Council. Recommended for acceptance by X. Yu. (Corresponding author: Jiaman Ding, Xiuxing Li.)

Lianyin Jia, Yongwang Miao, Jiaman Ding, and Xiaodong Fu are with the Faculty of Information Engineering, Automation, Kunming University of Science, Technology, Kunming 650032, China (e-mail: lianyinjia@kust.edu.cn; miaoyw@stu.kust.edu.cn; jiamanding@kust.edu.cn; xiaodong_fu@hotmail.com).

Suprio Ray is with the Faculty of Computer Science, University of New Brunswick, Fredericton, NB E3B 5A3, Canada (e-mail: rray@unb.ca).

Xiuxing Li is with the School of Computer Science & Technology, Beijing Institute of Technology, Beijing 100811, China (e-mail: xxli@bit.edu.cn).

Digital Object Identifier 10.1109/TKDE.2026.3699763

Spatial keyword query combines spatial range queries and keyword queries, aiming to retrieve all objects within a given spatial range and containing all query keywords. Indexing is one of the most important keys to achieving efficient spatial keyword queries. Existing spatial keyword indexes typically involve building spatial indexes like R-trees or Quadtrees, as well as text indexes such as inverted indexes or bitmap indexes. These indexes are then combined using strategies like spatial-first or text-first approaches.

Chen et al. [2] evaluated 12 indexes that support spatial keyword queries, and their findings indicate that SFC-Quad [3] exhibits relatively superior query performance. SFC-Quad utilizes a spatial-first index that combines Quadtree and inverted index. A Quadtree indexes the spatial components of objects, while an inverted index processes the textual components. Each keyword corresponds to an inverted list that stores all the identifiers (*IDs*) of objects containing that keyword. The *IDs* in each inverted list are sorted in Z-order curve order. During query execution, SFC-Quad first performs a coarse-grained spatial range query using the Quadtree to obtain the Minimum Containing Node (*MCN*) that includes the query bounding box, thereby obtaining an *ID* Range (*IDR*) associated with that node. Then, based on the *IDR*, a list intersection algorithm is executed on the corresponding inverted files to obtain the query candidates.

The limitation of SFC-Quad is that it treats all keywords equally, ignoring differences in their distributions. This results in low efficiency when keyword distributions are skewed. To address this issue, Jia et al. [4] proposed ARM-SQ, which utilizes association rule mining to mine frequent itemsets and materializes the intersection results of frequent itemsets, effectively avoiding the inefficiency of long list intersections.

Both SFC-Quad and ARM-SQ exploit the direct relationship between Quadtree and Z-order curve [5]. Specifically, partitioning objects in a Quadtree is equivalent to partitioning the *IDs* in the inverted lists sorted in Z-order, so the *IDR* return by Quadtree can be directly used to locate the range in the inverted lists. As a result, these two algorithms exhibit high efficiency. Even so, they still have the following significant weaknesses. 1) Both algorithms are Z-order curve based. It is well-known that Z-order curve suffers from large jumps, leading to decreased query efficiency [6], [7], [8], [9]. 2) The spatial part of these algorithms returns a large *IDR* corresponding to the *MCN*, resulting in many objects outside the query box being returned. Consequently, we refer to the *IDs* of these objects as irrelevant *IDs*. 3) The

above algorithms use Document-At-A-Time algorithm [10] to perform list intersections, leading to a large number of unnecessary *ID* accesses and thus reducing the efficiency of keyword queries.

To address these issues, we introduce a novel and efficient index, HQT-TI, which consists of two components: HQT and TI. The first component, HQT, is a spatial index that combines the Hilbert curve with a Quadtree. HQT organizes Quadtree nodes along the Hilbert curve to establish the direct relationship between HQT and Hilbert curve. We adopt the Hilbert curve instead of the Z-order curve primarily due to its superior clustering properties, which brings us two major benefits: 1) With the Hilbert curve, more *IDRs* can be merged during subsequent spatial range queries, leading to a significant reduction in the number of returned *IDRs*. 2) The *IDs* in the inverted lists are more tightly clustered, which substantially reduces the number of IO operations. Building upon HQT, we developed an efficient algorithm, HQT-SQ, for spatial range queries. HQT-SQ is also a coarse-grained spatial query processing approach. However, it employs query drill-down to decrease the irrelevant *IDs* and IO accesses. Besides, HQT-SQ employs a limited breadth expansion search strategy to effectively reduce the number of overlap checks between HQT nodes and query windows. The second component is TI, a trie and inverted index hybrid index. Based on TI, we developed a segmented list intersection algorithm, SLI-KQ, to significantly improve keyword query efficiency. Finally, by integrating HQT-SQ and SLI-KQ, we developed a highly efficient spatial keyword query algorithm, HS-SKQ. Extensive experimental results demonstrate that HS-SKQ outperforms SFC-Quad by up to two orders of magnitude and ARM-SQ by up to $5.46\times$.

II. PRELIMINARY

A. Basic Definitions

Given a spatial-textual dataset $\mathcal{D}$ consisting of a series of objects, each object $o \in \mathcal{D}$ is represented as $o = \langle o.P, o.W \rangle$, where $o.P$ represents the two-dimensional coordinates of object o , and $o.W$ denotes the set of keywords contained in o . Each object o has a unique identifier $o.ID$. A query q is also composed of two parts: $q = \langle q.R, q.W \rangle$, where $q.R$ represents a spatial query bounding box defined by its bottom-left coordinate $\mathcal{P}_1 = (x_1, y_1)$ and top-right coordinate $\mathcal{P}_2 = (x_2, y_2)$, and $q.W$ denotes the set of keywords associated with q .

Definition 1. Spatial Keyword Query: Given a spatial-textual dataset $\mathcal{D}$ and a query $q = \langle q.R, q.W \rangle$, a spatial keyword query returns all objects o in $\mathcal{D}$ that satisfy the following two conditions: 1) $o.P$ in $q.R$; 2) $q.W \subseteq o.W$

Example 1: Given the example dataset shown in Table I and its corresponding schematic diagram in Fig. 1, a query q corresponds to $q.R = \langle (3, 2), (6, 3) \rangle$ and $q.W = \{a, b, c, e\}$ (the query bounding box is depicted by the red rectangle in Fig. 1). Executing spatial keyword query will return object 2 as the final result. Although objects 10 and 11 are also within the query box, they do not contain all the query keywords, and although object 1 contains all the query keywords, it is not within the query box.

TABLE I
SAMPLE DATASET

$o.ID$	$o.P$	$o.W$	$o.ID$	$o.P$	$o.W$
0	(0,0)	$\{a,b,c,g\}$	7	(3,5)	$\{a,c,e\}$
1	(2,2)	$\{a,b,c,d,e\}$	8	(6,5)	$\{f,g\}$
2	(3,3)	$\{a,b,c,d,e\}$	9	(7,2)	$\{a,b,c,d,f\}$
3	(0,2)	$\{a,f,h\}$	10	(5,3)	$\{b,e,f,h\}$
4	(0,4)	$\{b,g\}$	11	(4,2)	$\{a,c,d\}$
5	(1,6)	$\{a,c\}$	12	(7,0)	$\{a,b\}$
6	(3,7)	$\{c,b,d\}$			

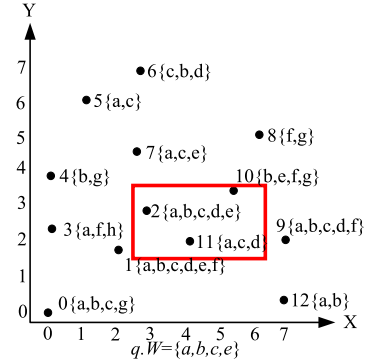


Fig. 1. Distribution corresponding to data in Table I.

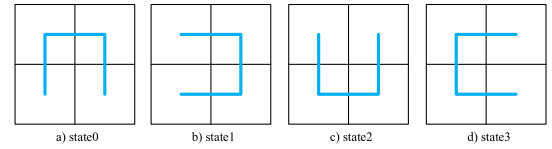


Fig. 2. Four orientations of Hilbert curve of order 1.

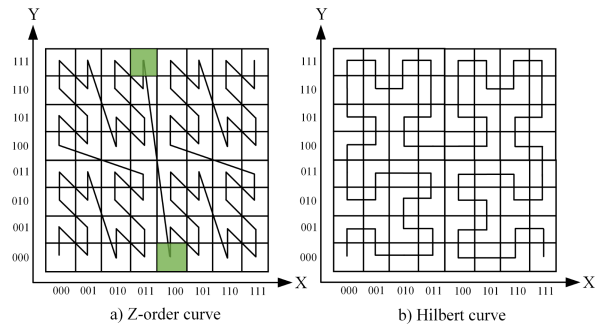


Fig. 3. Z-order curve of order 3 and Hilbert curve of order 3.

B. Space-Filling Curve

For two-dimensional space, the Hilbert curve of order 1 has four different orientations (states) as illustrated in Fig. 2: 0 (opening downward), 1 (opening leftward), 2 (opening upward), and 3 (opening rightward). The entire Hilbert space can be further divided into four quadrants, with a Hilbert curve of order 1 (with corresponding rotations and flips) placed in each quadrant to obtain a Hilbert curve of order 2. This iterative process continues to generate curves of any order.

The schematic diagrams of a Z-order curve of order 3 and a Hilbert curve of order 3 are shown in Fig. 3(a) and (b),

respectively. From these diagrams, it is visually apparent that the Z-order curve exhibits greater jumps compared to the Hilbert curve. On the Z-order curve, adjacent points may correspond to spatial positions that are far apart (as indicated by the two green background squares in Fig. 3(a), whereas for the Hilbert curve, adjacent points on the curve always correspond to adjacent positions in space.

C. Related Work

With the widespread proliferation of location-based devices and services, spatial keyword query has become increasingly important. Over the past decade, a significant number of spatial keyword query algorithms have been proposed. A spatial keyword query combines a spatial range query with a keyword query. Most algorithms utilize spatial indexes such as R-tree and their variants [11], [12], [13], [14], Quadtree [3], [4], [15], or KD-tree [16] for spatial components, and inverted indexes or bitmaps for textual components. The integration of spatial and textual components can be achieved through spatial-first, text-first, or tightly integrated spatial-textual approaches. Comprehensive reviews in this field can be found in references [17], [18]

Early research on spatial keyword query predominantly focused on R-tree based algorithms. IF-R* [13] integrates R*-trees with inverted index in a text-first fashion, building an R*-tree for each individual keyword in the dataset. IR-tree [11] tightly integrates R-trees with inverted index, where each node not only stores its corresponding inverted list but also the textual summaries of all objects in the subtree. Building upon IR-tree, CIR-tree [14] enhances efficiency by clustering objects based on their textual components. WIR-tree [12] iteratively groups objects based on the most frequent elements. Approaches [19], [20] map both spatial and textual dimensions to a two-dimensional space and then use R-tree indexing on the mapped data. However, their keyword queries are focused on similarity rather than exact matches.

Apart from R-trees and their variants, indexes like Quadtree and KD-tree are widely used in spatial keyword queries. SFC-Quad [3] is a typical Quadtree based representative. It initially sorts the objects in $\mathcal{D}$ in Z-order curve order, then creates a Quadtree for the spatial component and an inverted index for the textual component. It leverages the direct relationship between Quadtree and Z-order curve to accelerate queries. Similar to SFC-Quad, ARM-SQ [4] also combines Quadtree and inverted index. However, it differs in that it constructs materialized inverted indexes based on the frequent itemsets obtained from association rule mining algorithms, effectively improving the efficiency of spatial keyword query algorithms under skewed text distributions. A compact index called cBiK is proposed in [16], which reduces index space by utilizing KD-Tree and compressed bitmaps.

Recently, the paradigm of learned indexes has been extended to spatial and spatio-textual domains, giving rise to several learned spatial keyword indexes. Ding et al. [21] proposed LSTI, a learned spatial-textual index that combines RadixSpline with inverted lists, using Z-order curve to map spatial data into one dimension. Sheng et al. [22] proposed WISK, a workload-aware

learned index for spatial keyword queries that optimizes data partitions and hierarchical structures by learning from query distributions. Despite their merits, LSTI suffers from the limited clustering property of the Z-order curve. Moreover, none of the above algorithms consider skewed textual distributions, resulting in suboptimal efficiency under such scenarios.

In addition, spatial keyword queries have been widely extended into new domains. Including privacy-preserving spatial keyword queries [23], [24], [25], continuous spatial keyword queries [26], [27], spatial keyword queries on Road Networks [28], top-k spatial keyword queries [29], spatial-temporal keyword queries [30], semantic spatial keyword query [31], [32], [33]. Despite numerous efforts, existing algorithms suffer from drawbacks such as large jumps in Z-order curve, coarse-grained query leading to many irrelevant *IDs*. We address these issues by introducing a novel index that combines Hilbert curve and Quadtree. Through spatial query drill-down and segmented list intersection, it significantly enhances the efficiency of spatial keyword query algorithms.

III. SPATIAL INDEX AND QUERY ALGORITHM

As previously discussed, existing Z-order curve based algorithms suffer from large jumps and coarse-grained queries leading to large irrelevant *IDs*. In facing this, we have designed an efficient spatial index HQT-TI, with two components HQT and TI. We describe index, HQT, and the corresponding spatial query algorithm, HQT-SQ.

A. HQT

The existing Z-order curve based spatial keyword query algorithms [3], [4] commonly employ Quadtree for spatial indexing. For the textual part, they create inverted indexes on objects sorted in Z-order curve order. The core idea is to leverage the natural consistency between Quadtree and the Z-order curve, allowing the *IDR* obtained from Quadtree to directly apply to the inverted index.

The main drawback of the Z-order curve is its large jumps, resulting in returning a large number of *IDs* that are not within the query bounding box during spatial queries. In contrast, the Hilbert curve exhibits better clustering properties. Nearby objects are usually closer in the mapped one-dimensional space of the Hilbert curve than that of the Z-order curve. Therefore, utilizing the Hilbert curve can effectively enhance the performance of spatial keyword queries. However, unlike the Z-order curve, there is no direct relationship between the Hilbert curve and Quadtree, so we cannot directly use a Quadtree to support the Hilbert curve.

To address these issues, we propose a novel integrated Hilbert curve and Quadtree index, HQT. Similar to Quadtree, the root node of HQT represents the entire data space, and each non-leaf node N contains four children, representing the quadrants of the space corresponding to N .

HQT maps quadrants of the Hilbert space to its nodes. To build the direct relationship between HQT and the Hilbert curve, unlike Quadtree, each node in HQT is laid out along the Hilbert curve. As the Z-order curve has only one state, each non-leaf

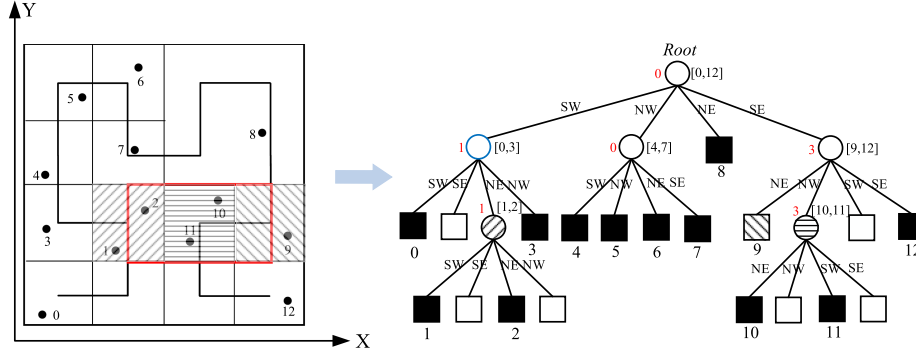


Fig. 4. HQT created on dataset in Table I.

node in Quadtree always corresponds to four children representing the southwest (SW), northwest (NW), southeast (SE), and northeast (NE) quadrants of a Z-order space. In contrast, the Hilbert curve has four states, where the arrangement of four children nodes varies depending on the state of the parent node. To achieve this, we extend each node in HQT with a state attribute to store its current Hilbert state. During the construction of HQT, the sequence of the four children nodes of N is determined based on the state of N . For example, in Fig. 4, the state of the root N is 0, so its four child nodes are organized in the sequence of the four quadrants of the Hilbert curve: SW, NW, NE, SE, in line with Fig. 2(a). The state of the root's first child, $N.N_1$, is 1, so its four child nodes are organized in the sequence of SW, SE, NE, NW, in line with Fig. 2(b). This ensures that each child node $N.N_i$ of node N corresponds to the i th quadrant of the Hilbert space associated with N , establishing consistency between each HQT node and the Hilbert curve. Based on the description above and the dataset outlined in Table I, the HQT index organization is illustrated in Fig. 4.

As seen in Fig. 4, a key advantage of HQT is that by traversing its nodes in a depth-first manner, one can obtain the objects in Hilbert curve order.

In Fig. 4, circular nodes represent non-leaf nodes, rectangular nodes filled in black indicate leaf nodes containing objects, and unfilled rectangular nodes denote empty leaf nodes. For clarity, we define the state of the root node as always being 0, corresponding to the first order always opening downwards. For any current node N , $N.State$ denotes its current Hilbert state (as indicated by the red numbers to the left side of the nodes in Fig. 4). Each node N corresponds to an *IDR*, as shown to the right side of the nodes. $N.IDR[0]$ and $N.IDR[1]$ respectively represent the minimum and maximum *ID* of objects contained within N . Additionally, $N.Region$ denotes the region represented by the node, $N.IsLeaf$ indicates whether N is a leaf node.

Example 2: As shown in Fig. 4, the four children of the root node divide the entire Hilbert space into four quadrants, with each node representing a quadrant of the entire Hilbert space. As the state of the root node is 0, its four children are arranged in the order of SW, NW, NE, and SE. The first child node of the root node (blue circular node) corresponds to the bottom-left quadrant of the Hilbert space. Since the state of this node is 1, its four children are arranged in the order of SW, SE, NE, and NW.

Complexity Analysis: The construction of HQT consists of three main steps: 1) Hilbert code computation: Assuming a Hilbert curve of order k , we adopt the efficient group-based encoding algorithm with grouping parameter g [9]. Encoding a single coordinate requires $O(k/g)$ time; thus, encoding n coordinates results in a time complexity of $O(nk/g)$. 2) Hilbert code sorting: Sorting the n Hilbert codes requires $O(n \log n)$ time. 3) HQT construction: The construction of the HQT index has a complexity similar to that of a standard Quadtree, i.e., $O(n \log n)$. Therefore, the overall time complexity is $O(nk/g + n \log n)$. The main space overhead of HQT is comparable to that of a standard Quadtree, yielding a space complexity $O(n)$.

B. Spatial Query Algorithm Based on HQT

SFC-Quad and ARM-SQ return an *IDR* corresponding to Minimum Containing Node (*MCN*) during spatial range queries. The *IDs* in this *IDR* may be very large, increasing the cost of subsequent text queries.

Given the query region $q.R$ represented by the red rectangular box in Fig. 4, the *MCN* containing this query region is the root node, corresponding to the entire Hilbert space. Although objects with *IDs* 0-1, 3-9, and 12 are not within the query box, they are still returned as results of the spatial range query, leading to an increase in subsequent processing costs.

Intuitively, if the query can be drilled down, the number of irrelevant objects returned can be effectively reduced, thereby improving efficiency. For ease of description, we refer to the layer corresponding to the *MCN* as level l_0 . Subsequent layers resulting from drill-down are labeled as $l_1, \dots, l_m$ based on their drill-down depths.

Query drill-down requires performing an overlap check between the query box and the child nodes of the *MCN* to determine whether $q.R$ intersects with these children nodes. Intuitively, this can be done by directly performing overlap checks between the query bounding box and each of the four child nodes of the *MCN*. However, this approach requires four separate overlap checks, which is quite costly.

Based on the analysis of query boxes, we found that when drilling down to layer l_1 , for the *MCN* node N , the overlap between $q.R$ and the child nodes of N adheres to the following two properties: 1) Due to the characteristics of the rectangular query box, $q.R$ cannot overlap with only three child nodes of N .

Algorithm 1: FSFO

Input : $q.\mathcal{R}$: the spatial part of q
 N : the *MCN* corresponding to $q.\mathcal{R}$
Output: O : the overlapped child nodes of N ;

```

1  $f_1 = \text{Overlap}(N.N_1, q.\mathcal{R})$ 
2  $f_2 = \text{Overlap}(N.N_2, q.\mathcal{R})$ 
3 if  $f_1 == 1$  and  $f_2 == 1$  then
4    $f_4 = f_3 = \text{Overlap}(N.N_3, q.\mathcal{R})$ 
5 else
6    $f_3 = \bar{f}_1$ 
7    $f_4 = \bar{f}_2$ 
8  $O \leftarrow$  the overlapped child nodes of  $N$  with  $f_i = 1$ ;
```

2) Similarly, $q.\mathcal{R}$ cannot overlap with only one child nodes of N ; otherwise, N could not be the *MCN*. Leveraging these properties, we propose a novel First and Second nodes First Overlap (FSFO) check algorithm, as depicted in Algorithm 1.

In Algorithm 1, $\text{Overlap}(N.N_i, q.\mathcal{R})$ checks the overlap check between $N.N_i$ and $q.\mathcal{R}$, and the flag f_i is set to 1 if they overlap, and 0 otherwise. When f_1 and f_2 are not both 1 (corresponding to the cases $f_1f_2 = 00, 10$, and 01), it is observed that f_3 is always opposite to f_1 , and f_2 is always opposite to f_4 . Therefore, employing logical NOT operations directly reduces the number of overlap checks to expedite efficiency (lines 5-7).

Example 3: We use the Hilbert curve illustrated on the left side of Fig. 4 to describe the overlap relationships. The *MCN* is the root node, which corresponds to the entire Hilbert region. Its four sub-regions along the Hilbert curve: SW, NW, NE, and SE, correspond to the four child nodes of the root. After performing overlap checks between $q.\mathcal{R}$ and the SW and NW sub-regions, we obtain the flag bits $f_1f_2 = 10$. Leveraging the properties stated above, we deduce that $q.\mathcal{R}$ does not overlap with the NE sub-region, while it must overlap with the SE sub-region. Consequently, the flags for the remaining sub-regions can be derived directly by bit-inverting f_1f_2 , yielding $f_3f_4 = 01$. Thus, only two overlap checks are required in this case.

The number of overlap checks required in FSFO satisfies Theorem 1.

Theorem 1: Under an equal probability distribution of query boxes, the expected number of overlap checks for the four children nodes of the *MCN* is 2.4.

Proof: It is evident that the possible query box overlaps include five cases: $f_1f_2f_3f_4 = 1111, 1100, 1001, 0110$ and 0011 .

For $f_1f_2f_3f_4 = 1111$ and 1100 , each requires three overlap checks. For each of the other three cases, only two overlap checks are needed. Therefore, the expected number of overlap checks is calculated as $(3 \times 2 + 2 \times 3)/5 = 2.4$. Thus, Theorem 1 holds.

When drilling down to layer l_1 , only 2.4 overlap checks are needed on average, significantly reducing the number of checks. However, when further drilling down to layer l_2 and deeper, considering that a node N at layer l_1 is a *non-MCN* node, it may only have one child node overlapping with $q.\mathcal{R}$. For example, as shown in Fig. 4 when drilling down to layer l_2 , only one child node (filled with left diagonal lines) of the blue circular node N overlaps with $q.\mathcal{R}$. Therefore, the aforementioned property

Algorithm 2: FSFO⁺

Input : $q.\mathcal{R}$: the spatial part of q
 N : a node overlapped with $q.\mathcal{R}$
Output: O : the overlapped child nodes of N

```

1  $f_1 = \text{Overlap}(N.N_1, q.\mathcal{R})$ 
2  $f_2 = \text{Overlap}(N.N_2, q.\mathcal{R})$ 
3 if  $f_1 == 1$  and  $f_2 == 1$  then
4    $f_4 = f_3 = \text{Overlap}(N.N_3, q.\mathcal{R})$ 
5 else if  $f_1 == 0$  and  $f_2 == 0$  then
6    $f_3 = \text{Overlap}(N.N_3, q.\mathcal{R})$ 
7   if  $f_3 == 1$  then
8      $f_4 = \text{Overlap}(N.N_4, q.\mathcal{R})$ 
9   else
10     $f_4 = 1$ 
11 else if  $f_1 == 1$  and  $f_2 == 0$  then
12    $f_3 = 0$ 
13    $f_4 = \text{Overlap}(N.N_4, q.\mathcal{R})$ 
14 else
15    $f_3 = \text{Overlap}(N.N_3, q.\mathcal{R})$ 
16    $f_4 = 0$ 
17  $O \leftarrow$  the overlapped child nodes of  $N$ 
```

2) no longer applies. Hence, we propose the FSFO⁺ algorithm as shown in Algorithm 2 to support drilling down into layers deeper than 1.

Example 4: Further drilling down to the first child node of the root, whose opening orientation is leftward. Its four sub-regions along the Hilbert curve: SW, SE, NE, and NW, correspond to its four child nodes. After performing overlap checks between $q.\mathcal{R}$ and the SW and SE sub-regions, we obtain the flag bits $f_1f_2 = 00$. This indicates that at least one, or both, of the remaining two sub-regions are overlapped by $q.\mathcal{R}$. We then proceed to check the NE sub-region, obtaining $f_3 = 1$, and subsequently check the NW sub-region, obtaining $f_4 = 0$. In this case, a total of four overlap checks are required.

In Algorithm 2, following steps similar to Algorithm 1, the calculation of the number of overlap checks can be divided into the following three cases:

- 1) When $f_1f_2 = 11$ (corresponding to $f_1f_2f_3f_4 = 1111$ or 1100), an additional $N.N_3$ need to be checked, totaling three overlap checks.
- 2) When $f_1f_2 = 00$, $N.N_3$ need to be checked. For the case $f_3 = 0$ (corresponding to $f_1f_2f_3f_4 = 0001$), only three overlap checks are needed. For the case $f_3 = 1$ (corresponding to $f_1f_2f_3f_4 = 0010$ or 0011), $N.N_4$ need to be further checked, totaling four checks.
- 3) When $f_1f_2 = 10$ (corresponding to $f_1f_2f_3f_4 = 1000$ or 1001) or $f_1f_2 = 01$ (corresponding to $f_1f_2f_3f_4 = 0100$ or 0110), $N.N_3$ or $N.N_4$ need to be checked, each requiring three checks.

Therefore, the expected number of overlap checks is $(3 \times 7 + 4 \times 2)/9 = 3.22$.

Theoretically, we can extend the spatial queries to the finest granularity. However, overly fine query granularity will result in a large number of smaller *IDRs*, thereby increasing spatial query costs and subsequent text processing overheads. Therefore, we

only extend spatial queries to a limited m layers to balance between query cost and filtering capability.

For tree based queries, depth-first search is the simplest and the most common way. However, simple depth-first search requires independent overlap checks for all child nodes of node N , which makes it difficult to leverage the rapid overlap checking advantages of FSFO and FSFO⁺. To address this issue, we propose a HQT based spatial range query algorithm, HQT-SQ. HQT-SQ using a depth-first search with limited breadth expansion strategy, aiming to minimize the number of overlap checks during the query processing at a relatively low storage cost.

HQT-SQ extends the traditional depth-first search algorithm with limited breadth expansion. Specifically, when visiting $N.N_1$ of node N , instead of continuing to recursively search the children of $N.N_1$, HQT-SQ performs overlap checks for all children nodes of N using FSFO or FSFO⁺. This approach aims to avoid subsequent independent overlap checks for each child node. Considering that multiple child nodes of N may overlap with $q.R$ after breadth expansion, we introduce a stack S to store the child nodes of N that overlap with $q.R$. During the query processing, these children nodes are recursively pushed into S in reverse order.

While a fixed-depth drilling strategy (i.e., drilling all nodes down to a fixed depth m) is straightforward, it may lead to performance degradation under skewed data due to the significant variance in ID counts across nodes. Therefore, to address this issue, an adaptive drilling strategy is necessary to evaluate the drilling benefit for each node on an individual basis.

To address this issue, a corresponding cost model is required. For external-memory spatial keyword query algorithms, the IO cost incurred by textual index access constitutes the most significant performance bottleneck. As our proposed algorithm adopts a spatial-first filtering strategy, it is challenging to precisely quantify the IO cost during the spatial query phase. Considering this, we utilize the number of $IDRs$ (reflects the degree of randomness in IO operations) and the number of IDs (represents the volume of data transferred) as proxies for IO cost. Based on these metrics, we introduce an adaptive heuristic to determine whether to further drill down a given node N . The heuristic is defined as follows: if $|N.IDR[1] - N.IDR[0]| > \rho$, the node continues to be drilled down; otherwise, it is returned as a result. Here, ρ is a preset threshold representing the average ID density per IDR , which can be statistically derived from the ratio $\#AID/\#IDR$ over a large number of queries. Here, $\#IDR$ and $\#AID$ denote the number of $IDRs$ and the average number of IDs returned per spatial query, respectively. To prevent a large drill-down depth caused by the skewed data distributions, we further impose a maximum depth threshold δ . Drilling down for any node ceases once this depth limit is reached.

The detailed HQT-SQ is outlined in Algorithm 3.

Algorithm 3 first locates the MCN and pushes it into stack S with an initial depth of 0 (lines 3-4). While S is not empty, if the node N corresponding to the top of S satisfies the following four conditions: 1) the drilling depth of N is greater than δ ; 2) $N.IDR[1] - N.IDR[0] < \rho$; 3) N is a leaf node; 4) N is contained within $q.R$, then further drilling down is unnecessary, and $N.IDR$ becomes a result satisfying the spatial query

Algorithm 3: HQT-SQ

Input : $q.R$: the spatial part of q ,
 m : the maximum depth need to be drilled
Output: I : all the $IDRs$ satisfying the spatial query

```

1  $S \leftarrow$  an empty stack
2  $maxID \leftarrow 0, I \leftarrow \emptyset$ 
3 find the  $MCN$   $N$  containing  $q.R$  following the path from
   root to  $N$ 
4  $S.push(N, 0)$ 
5 while  $S$  is not empty do
6    $(N, d) \leftarrow S.pop()$ 
7   if  $d > \delta$  or  $N.IDR[1] - IDR[0] < \rho$  or
      $N.IsLeaf == true$  or  $N$  is contained in  $q.R$  then
8     if  $N.IDR[0] - maxID == 1$  then
9       Merge  $N.IDR$  with the last  $IDR$  in  $I$ 
10       $maxID \leftarrow N.IDR[1]$ 
11     else
12        $I \leftarrow I \cup N.IDR$ 
13        $maxID \leftarrow N.IDR[1]$ 
14   else
15     if  $d = 0$  then
16        $ONs \leftarrow FSFO(q.R, N)$ 
17     else
18        $ONs \leftarrow FSFO^+(q.R, N)$ 
19     for  $i$  from  $|ONs|$  to 1 do
20        $S.push(ONs[i], d + 1)$ 
```

(lines 5-13). Benefiting from the Hilbert curve order of nodes in HQT, if $N.IDR[0] - maxID = 1$, then $N.IDR$ can be directly merged into the last IDR in I (lines 8-10). This feature allows HQT-SQ to directly obtain compact $IDRs$ during the query processing without needing subsequent independent IDR merging processes. During the drill-down process, FSFO or FSFO⁺ is invoked based on the current drill-down depth to obtain child nodes overlapping with $q.R$ (lines 14-18), and these child nodes along with their drill-down depth are pushed into S in reverse order (lines 19-20). After the algorithm finishes, I stores all $IDRs$ satisfying the query.

Example 5: As illustrated in Fig. 4, for a given query $q.R = \langle (3, 2), (6, 3) \rangle$, the MCN is identified as the root node N , with its corresponding IDR being $[0, 12]$. Since N does not satisfy the condition specified in line 7 of Algorithm 3, we drill down to its child nodes at layer l_1 . Using the FSFO strategy outlined in line 16, nodes $N.N_1$ and $N.N_4$ are found to overlap with $q.R$, yielding the $IDRs$ $[0, 3]$ and $[9, 12]$, respectively. Following a similar approach, further drilling down $N.N_1$ to layer l_2 yields the IDR $[1, 2]$, while drilling down $N.N_4$ produces the $IDRs$ $[9, 9]$ and $[10, 11]$. Since the maximum ID of the second IDR is adjacent to the minimum ID of the third, they can be merged, resulting in the final $IDRs$ $[1, 2]$ and $[9, 11]$.

It is worth noting that the limited breadth expansion we employ only expands the children nodes of N . Although we could adopt a fully breadth-first traversal strategy (i.e., traversing HQT layer by layer), doing so would increase query time and storage space. The reason being: when expanding to depth δ , in the worst case (corresponding to a full 4-ary tree), a fully

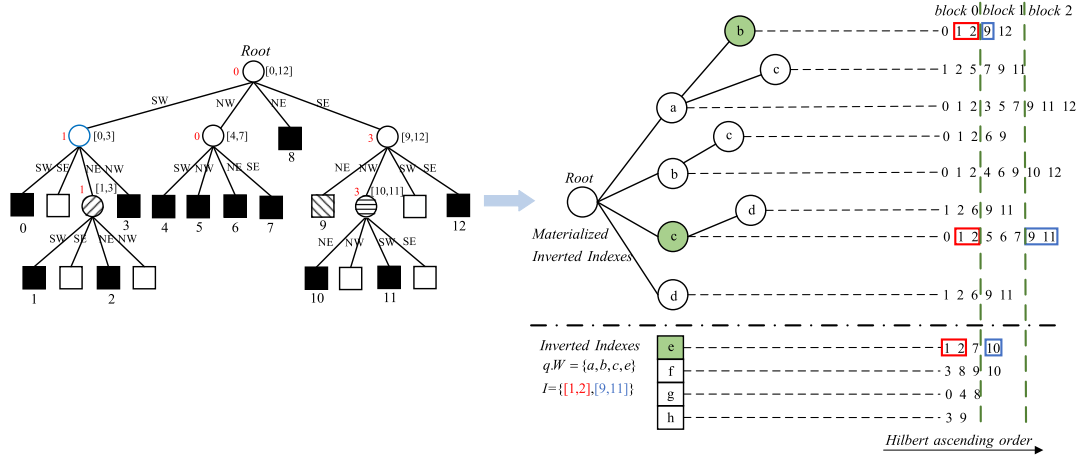


Fig. 5. HQT-TI created on dataset in Table I.

breadth-first strategy would require pushing $4^1 + 4^2 + \dots + 4^{\delta-1} = \frac{4 \times (4^{\delta-1} - 1)}{3}$ nodes into S , resulting in exponential space complexity. In contrast, HQT-SQ only needs to push up to four nodes per layer from the 1st to the $(\delta-1)$ th layer, hence requiring a maximum of $4(\delta-1)$ nodes pushed into S . This significantly reduces the space overhead compared to a fully breadth-first search.

HQT-SQ combines the advantages of depth-first search and breadth-first search and allows for naturally merging the $IDRs$ obtained during the search process. Therefore, it achieves high querying efficiency.

Complexity Analysis: The main cost of HQT-SQ lies in the overlap checks performed during drilling down process. Considering that FSFO and FSFO⁺ evaluate overlap check in a group of 4 nodes, we analyze the complexity in terms of groups. When the first layer, it requires 2.4 overlap checks per group. For the other layers, in the worst case, from layer l_2 to layer l_δ , a total of $4^1 + 4^2 + \dots + 4^{\delta-1} = \frac{4 \times (4^{\delta-1} - 1)}{3}$ groups of overlap checks are needed. Therefore, the total number of overlap checks required is $2.4 + \frac{4 \times (4^{\delta-1} - 1)}{3} = 1.1 \times 4^\delta - 2$.

Although the above number indicates exponential complexity, it should be noted that: 1) in most cases, HQT is typically not complete 4-ary tree; 2) as demonstrated in subsequent experiments, the drill-down depth is usually small. Therefore, the actual time cost of spatial queries in practice is often not high.

IV. TEXT INDEX AND KEYWORD QUERY ALGORITHM

A. Association Rule Based Text Index

Most existing algorithms treat all keywords equally, ignoring the differences in keyword frequency distribution. Frequent words correspond to long lists, and when there are many frequent words in a query, the intersection of these long lists will significantly reduce query efficiency. To address this issue, we follow the approach in [4] to construct a hybrid index combining trie and inverted index (TI). Given a support threshold δ , we first use association rule mining algorithms to mine frequent itemsets from the text dataset and index these itemsets using a

trie. Each frequent itemset f_i corresponds to a path in the trie, where the last element of f_i points to an inverted list containing object IDs composed of all elements in f_i . These inverted lists can be viewed as materialized intersections of lengthy lists, effectively mitigating the problem of lengthy list intersections. For non-frequent elements, inverted indexes are constructed directly. We assume that all inverted lists are stored on external storage.

Example 6: Given the text dataset¹ in Table I and a support threshold of $\delta = 5$, using association rule mining algorithms, we first obtain frequent itemsets $\{a\}$, $\{b\}$, $\{c\}$, $\{d\}$, $\{a, b\}$, $\{a, c\}$, $\{b, c\}$, $\{c, d\}$. Then we obtain TI by constructing a trie for these frequent itemsets and inverted lists for non-frequent elements $\{e, f, g, h\}$. The final combined index of HQT and TI, HQT-TI, is illustrated in Fig. 5.

Complexity analysis: Assuming that the average numbers of frequent and infrequent elements in each $o.W$ are $|o.W_f|$ and $|o.W_n|$, respectively, the number of corresponding frequent itemsets is at most $2^{|o.W_f|}$ in the worst case. Consequently, an object o can appear in at most $(2^{|o.W_f|} + |o.W_n|)$ inverted lists. Therefore, for n objects, the worst-case time and space complexities are both $O((2^{|o.W_f|} + |o.W_n|) * n)$. Although the above formulation includes an exponential term, in practice $|o.W_f|$ is relatively small, and the actual number of frequent itemsets is far less than $2^{|o.W_f|}$. As a result, the practical overhead remains moderate.

B. Segmented List Intersection Algorithm

After executing HQT-SQ, I stores all $IDRs$ obtained from Algorithm 3. For the objects within these $IDRs$, further integration with a keyword query algorithm is required to determine if they contain all keywords in $q.W$.

To effectively exploit the multiple disjoint IDR segments returned by HQT-SQ, we propose an efficient segment list intersection based keyword query algorithm (SLI-KQ). Specifically, SLI-KQ includes the following steps:

¹ To facilitate description, objects in this dataset have already been sorted according to the Hilbert curve order.

1) *Query Segmentation*: Considering TI combining trie and inverted lists, $q.W$ needs to be segmented to support queries in these two different structures. Based on the threshold δ , $q.W$ is divided into two parts: the frequent subset $q.W_f$ and the non-frequent subset $q.W_n$.

2) *Retrieving inverted lists corresponding to the query*: For $q.W_n$, we can directly obtain the inverted list corresponding to each element in $q.W_n$. For $q.W_f$, there are multiple possible materialized inverted list combinations in trie. To handle this, we introduce a heuristic rule. First, we select the frequent itemset f_i from $q.W_f$ that maximizes the materialized gain

$$G_{f_i} = \sum_{w \in f_i} |l_w| - |l_{f_i}|, \text{ where } |l_w| \text{ denotes the length of the}$$

inverted list corresponding to keyword w , and $|l_{f_i}|$ denotes the length of the materialized inverted list corresponding to f_i . Then, we recursively select f_i in the remaining $q.W_f - f_i$ part to maximize the materialized gain in a similar way, thereby obtaining the combination of materialized inverted lists that need to be accessed in the trie.

Ultimately, combining the materialized inverted lists for $q.W_f$ and the inverted lists for $q.W_n$ forms the inverted list collection L corresponding to $q.W$.

Example 7: Given $q.W = \{a, b, c, e\}$, through query segmentation, we obtain $q.W_f = \{a, b, c\}$ and $q.W_n = \{e\}$. After calculating the gain for the frequent itemsets $\{a, b\}$, $\{a, c\}$, $\{b, c\}$ corresponding to $q.W_f$, we find that $G_{ab} = |l_a| + |l_b| - |l_{ab}| = 12$ has the highest gain. Therefore, the selected inverted list collection L consists of l_{ab}, l_c, l_e .

3) *Segment List Intersection Algorithm*: For each IDR in I , we query the segments corresponding to that IDR in each list l in L , and perform list intersections on these segments. During execution, we employ heuristic rules to perform list intersections from shortest to longest segment length. Considering the efficiency of binary search in short list intersections [34], we utilize binary search to sequentially check if each ID in the shortest segment appears in other segments. If an ID does not appear in a particular segment, computations for that ID can be terminated early. This approach ensures high efficiency of the algorithm. The complete SLI-KQ algorithm is shown as Algorithm 4.

Algorithm 4 first partitions the query keyword set $q.W$ into a frequent subset $q.W_f$ and a non-frequent subset $q.W_n$ (line 2), based on which we extract inverted lists (lines 3-4) and materialized inverted lists (lines 5-7) to obtain L . Then, for each IDR returned by the HQT-SQ, we obtain its segments within L , sort these segments in ascending order of length, and add the IDs that appear in all segments to the candidate set C (lines 8-12).

Based on SLI-KQ, C stores the final candidates satisfying the keyword query. SLI-KQ demonstrates advantages in three aspects: 1) Avoiding access to a large number of irrelevant ID s by utilizing multiple small IDR s returned by HQT-SQ; 2) Reducing the costs of IO accesses and list intersections through frequent itemsets; 3) Further reducing the costs of IO accesses and list intersections through segment list intersection. Therefore, SLI-KQ achieves high efficiency.

Example 8: As shown in Fig. 5, given $I = \{[1, 2], [9, 11]\}$ returned by HQT-SQ, SLI-KQ performs segment intersection

Algorithm 4: SLI-KQ

Input : $q.W$: the textual part of q
 I : all IDR s returned from HQT-SQ
Output: C : the candidates

```

1  $C \leftarrow \emptyset, L \leftarrow \emptyset$ 
2  $q.W_f, q.W_n \leftarrow$  Split keywords based on  $\delta$ 
3 foreach  $n$  in  $q.W_n$  do
4    $L \leftarrow L \cup l_n$ 
5  $F \leftarrow$  all frequent itemsets selected using the heuristic rule to
   compute the maximized gain
6 foreach  $f_i \in F$  do
7    $L \leftarrow L \cup l_{f_i}$ 
8 foreach  $IDR$  in  $I$  do
9   Sort all segments corresponding to  $IDR$  in segment
   length ascending order
10  foreach  $ID$  in the shortest segment do
11    if  $ID$  in all the other segments then
12       $C \leftarrow C \cup ID$ 
```

Algorithm 5: HS-SKQ

Input : q : a query with the form $\langle q.R, q.W \rangle$
 m : the maximum depth need to be drilled
Output: RS : the final results

```

1  $I \leftarrow$  HQT-SQ( $q.R, m$ )
2  $C \leftarrow$  SLI-KQ( $q.W, I$ )
3  $RS \leftarrow \emptyset$ 
4 foreach  $o$  in  $C$  do
5   if  $o.P$  in  $q.R$  then
6      $RS \leftarrow RS \cup \{o\}$ 
```

algorithm on $L = \{l_{ab}, l_c, l_e\}$, as detailed in Example 7. SLI-KQ returns objects with ID 1 and 2 as candidates because their keywords contain all keywords of $q.W$. As illustrated in Fig. 5, using materialized inverted list l_{ab} requires only two IO accesses (assuming three elements correspond to one block), whereas accessing l_a and l_b directly would require six IO accesses. Additionally, by employing segment list intersection, we avoid accessing blocks in l_c that contain ID 5, 6, and 7, further reducing IO accesses.

Complexity Analysis: The primary overhead of SLI-KQ lies in segment list intersections. For an average segment length $\bar{s}$, the time complexity for binary searching across $|L|$ segments is $|L| \times \bar{s} \times \log_2 \bar{s}$. Thus, for the final $|I|$ IDR , the overall time complexity is $O(|I| \times |L| \times \bar{s} \times \log_2 \bar{s})$.

Notice that the IDR returned by HQT-SQ may include objects outside the query bounding box, so it is necessary to verify if the objects returned by SLI-KQ fall within $q.R$ to obtain the final results satisfying SKQ. The spatial keyword query algorithm HS-SKQ, achieved through combining HQT-SQ and SLI-KQ, is illustrated in Algorithm 5.

Example 9: Continuing from Example 6, given the candidates $C = \{1, 2\}$ returned by SLI-KQ, after verification, we find that the coordinate (2, 2) corresponding to ID 1 is not within $q.R$. Therefore, the final result is the object with ID 2.

TABLE II
SUMMARY OF KEYWORD DATASETS

PROPERTY	BMS	KOSARAK	FOURSQUARE
#sets	515597	990000	3680126
#unique words	1657	41270	266909
Max length	164	2498	10240
Min length	1	1	1
Average length	6.5	8.1	7.2

V. EXPERIMENT

All experiments in this study were conducted on a PC with an Intel i7-14650HX CPU@5.2 GHz and 48 GB of RAM. The operating system used was Windows 11, with the IntelliJ IDEA 2024 IDE and JDK version 1.8 for compilation.

A. Dataset

We use the following three real world datasets:

- 1) We combined the spatial dataset KGPOI² and the textual dataset BMS³ as a spatial-textual dataset referred to as DS1. KGPOI is a publicly available points of interest dataset from Kaggle, containing 442,405 coordinates. BMS is a clickstream dataset from the KDD Cup 2000 with 515,597 records. We randomly and without repetition assigned a record from BMS to each coordinate in Kaggle.
- 2) We combined the spatial dataset ROAD⁴ and the textual dataset KOSARAK⁵ as a spatial-textual dataset referred to as DS2. ROAD is a dataset of U.S. highway data with 20 million coordinates. KOSARAK is a clickstream dataset from Hungarian online news portals with 990,000 records. We randomly and without repetition assigned a coordinate from ROAD to each record in KOSARAK.
- 3) We used the global-scale check-in dataset collected by Foursquare⁶ referred to as DS3. The range of the data spans from April 2012 to September 2013. This dataset records the check-in activities of 266,909 users at 3,680,126 venues, making it a large-scale spatial-textual dataset with broad geographical coverage.

Details of the textual part of each dataset are provided in Table II.

B. Parameter Settings

To validate the performance of the proposed indexes and algorithms, we compare them with eight existing approaches, including six traditional spatial keyword indexes (IF-R* [13], WIR-Tree [12], IR-Tree [11], CIR-Tree [14], SFC-Quad [3], ARM-SQ [4]) and two learned spatial keyword indexes (LSTI [21] and WISK [22]). For each counterpart, we select their optimal parameter combinations to achieve its best performance. For R-tree and R*-tree based algorithms, we set page size to

32k and fanout to 4000. WIR-Tree iteratively divides objects in the dataset based on the most frequent words; thus, we set the number of frequent words for DS1, DS2, and DS3 to 5, 10, and 20, respectively. Similarly, for CIR-Tree, we set the number of clusters for DS1, DS2, and DS3 to 10, 20, and 40, respectively. For ARM-SQ and HS-SKQ, the minimum support thresholds for DS1, DS2, and DS3 are set to 400, 2000, and 5000, respectively. We set $\rho = 30000$ and $\delta = 6$ for HQT-SKQ in all the following experiments.

In this study, we evaluate the impact of parameters including drill-down depth m , number of high-frequency words X in query, total number of keywords Y a query, and query range percentage P (the proportion of a query bounding box over the dataset space) on query performance. We also assess various space-filling curve. For each experimental setup, we generate 1000 queries based on the specified parameters and report the total running time of these queries. Each experimental setup is executed 5 times, and the average running time is reported.

For detailed performance analyses, we evaluate algorithms such as SFC-Quad, ARM-SQ, and HS-SKQ based on three key metrics the number of *IDRs* returned for each query (#IDR), the Average number of *IDs* contained in the *IDRs* for each query (#AID), and the Average number of *IO* accesses for each query (#AIO).

C. Evaluation of the Adaptive Drilling Strategy

To evaluate the effectiveness of the adaptive drilling strategy, we compare it against the fixed-depth drilling approach, where a uniform depth m is applied to all nodes. In this experiment, we fix $X = 6$, $Y = 10$, $P = 10\%$, and vary m . The running times of HQT-SQ (spatial query component), SLI-KQ (keyword query component), and HS-SKQ (the overall spatial keyword query algorithm), along with their fixed-depth counterparts (suffixed with "-fixed"), across the three datasets are presented in Fig. 6.

As shown in Fig. 6, compared with HS-SKQ-fixed with $m = 0$ (corresponding to the counterpart without drilling down), HS-SKQ achieves a maximum efficiency improvement of $1.06\times$ across the three datasets. Besides, it is evident that the main time cost of HS-SKQ is attributed to SLI-KQ. We can reduce up to 52.6% of the keyword query cost at a relatively small spatial cost, indicating that drilling down can effectively enhance query efficiency. In this case, although #IDR of HS-SKQ increases slightly, #AID and #AIO decrease by up to 79.5% and 49.2%, respectively, contributing to the improvement in query performance.

Compared with HS-SKQ-fixed with optimal m , HS-SKQ achieves a maximum efficiency improvement of 36.9%. In this case, #AID, and #AIO of HS-SKQ decrease by up to 29.8%, and 11.9%, respectively, which fully demonstrates the advantage of HS-SKQ's adaptive drilling strategy over fixed-depth drilling counterpart.

D. Impact of the Number of Frequent Keywords X

To evaluate the impact of X on HS-SKQ, we fixed $Y = 10$, $P = 10\%$ and varied X from 2 to 10. Fig. 7 shows the results across the three datasets.

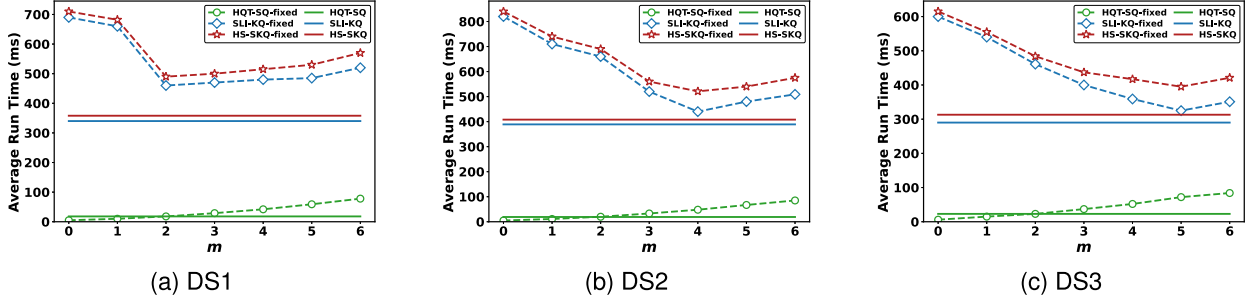
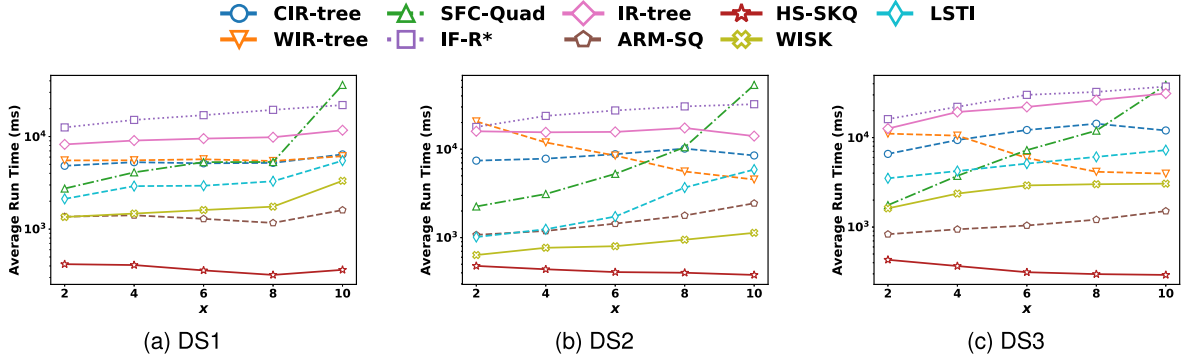
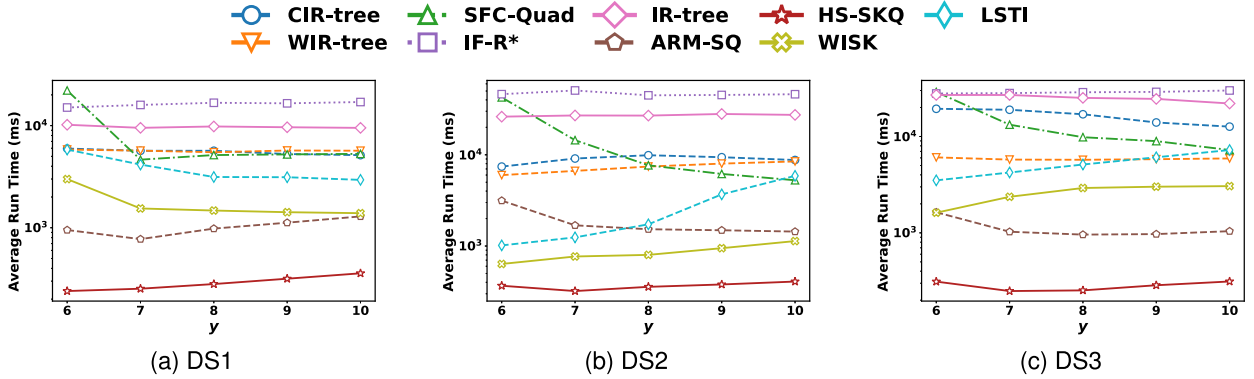
² <https://www.kaggle.com>

³ <https://www.kdd.org/kdd-cup/view/kdd-cup-2000/Data>

⁴ <http://www.diag.uniroma1.it/challenge9/download.shtml>

⁵ <http://fimi.uantwerpen.be/data>

⁶ <https://sites.google.com/site/yangdingqi/home/foursquare-dataset>

Fig. 6. Impact of m on query performance.Fig. 7. Impact of X on query performance.Fig. 8. Impact of Y on query performance.

From Fig. 7, it can be observed that the running times of most algorithms increase with X . However, the running time of HS-SKQ decreases as X increases because more materialized inverted lists are accessed with higher X , thereby reducing the cost of intersecting long lists. HS-SKQ exhibits significantly higher efficiency compared to other algorithms across all three datasets. Regarding the two learned-index-based algorithms, WISK demonstrates higher efficiency than LSTI. However, their overall efficiency is marginally lower than that of ARM-SQ, yet substantially lower than that of HS-SKQ. When $X = 10$, the performance improvement of HS-SKQ over SFC-Quad can be up to $139.46\times$ on the three datasets, respectively. Compared to its best competitor, ARM-SQ, the performance improvements of HS-SKQ can be $5.46\times$.

The key to improving query performance lies in that HQT-SQ can significantly reduce the #AID and #AIO through query drill-down, thereby lowering the cost of list intersection and improving the efficiency of SLI-KQ. For instance, at $X = 6$ on DS3, the #AID of HS-SKQ is only 28.4% of that of SFC-Quad and ARM-SQ, while the #AIO is only 4.3% of that of SFC-Quad and 31.7% of that of ARM-SQ. These reductions in #AID and #AIO lead to a substantial improvement in query efficiency.

E. Impact of the Total Keyword Count Y

To assess the impact of Y , we fixed $X = 6$, $P = 10\%$ and varied Y from 6 to 10. The results across the three datasets are shown in Fig. 8.

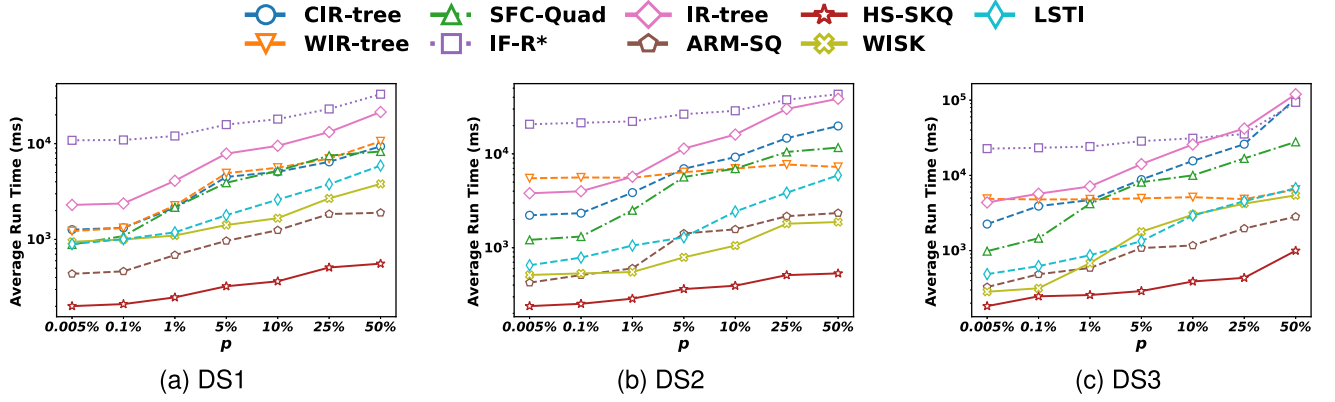
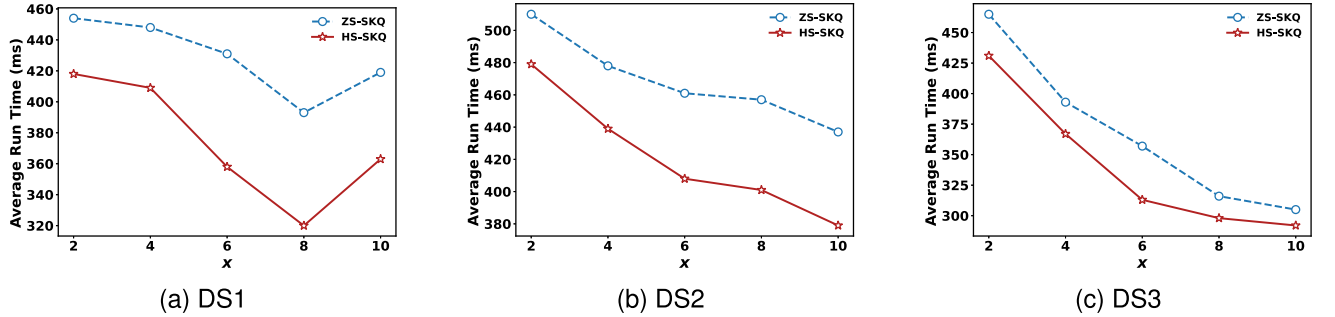
Fig. 9. Impact of P on query performance.

Fig. 10. Impact of different space-filling curves.

From Fig. 8, we observe that increasing Y has two main effects on query performance. On one hand, as Y increases, the number of query keywords in $q.W$ increases, leading to more inverted lists and thus increasing query costs. On the other hand, increasing Y also increases the number of short lists, which can enhance performance to some extent. Due to these dual factors, the runtime of each algorithm shows relatively stable changes with varying Y . Similarly, HS-SKQ outperforms the other algorithms across all datasets. At $Y = 6$, the performance improvements of HS-SKQ over SFC-Quad and ARM-SQ can be up to $115.07\times$ and $7.57\times$, respectively.

F. Impact of Query Region P

To evaluate the impact of P , we fixed $X = 7$, $Y = 10$, and varied P as 0.005%, 0.1%, 1%, 5%, 10%, 25%, and 50%. The experimental results on the three datasets are shown in Fig. 9.

From Fig. 9, it is evident that the running times for all algorithms increase with the increase of P . HS-SKQ demonstrates the optimal efficiency across different P , with its running time increasing less steeply with increasing P , indicating good scalability. This is because a larger P enhances the pruning effectiveness of HS-SKQ through drilling, which significantly reduces the number of objects that enter the keyword query phase, thereby leading to a greater reduction in subsequent text IO overhead.

G. Impact of Space-Filling Curves

To examine the impact of different space-filling curve on query performance, HS-SKQ was compared with its Z-order curve based counterpart. For clarity, Z-order curve based counterpart is denoted as ZS-SKQ in this paper. In the experiments, Y , and P were set to 10, and 10% respectively. The experimental results are shown in Fig. 10.

From Fig. 10, it is evident that across all three datasets, When X is 6, HS-SKQ outperforms ZS-SKQ, with performance improvements of 20.4%, 13.0%, and 14.1% respectively.

Although HS-SKQ and ZS-SKQ return the same #AID during spatial queries, there are key differences in: 1) More segments can be merged in HS-SKQ, resulting in a smaller #IDR. For example, on DS1, HS-SKQ generates only 2650 *IDRs*, significantly fewer than the 3810 *IDRs* generated by ZS-SKQ. 2) The coherence of HS-SKQ is better than that of ZS-SKQ can be expected, leading to fewer #AIO for HS-SKQ. Therefore, HS-SKQ demonstrates higher efficiency.

H. Index Size

Table III presents the index size across different datasets. CIR-tree requires a inverted file for each node, incurs the highest overall space overhead. IF-R* also incurs a larger space overhead, primarily because it create separate R*-tree for each keyword. The inverted lists of SFC-Quad, ARM-SQ and

TABLE III
INDEX SIZE

Index	DS1	DS2	DS3
CIR-tree	651.11MB	8.07GB	24.61GB
WIR-tree	66.79MB	583.05MB	2.13GB
SFC-Quad	21.74MB	68.91MB	153.39MB
IF-R*	212.51MB	2.79GB	17.32GB
IR-tree	83.83MB	724.07MB	2.32GB
ARM-SQ	175.03MB	269.13MB	314.41MB
WISK	76.87MB	374.99MB	1.04GB
LSTI	93.83MB	566.37MB	1.82GB
HS-SKQ	176.41MB	275.22MB	316.14MB

HS-SKQ are compressed using the OPT-PFD algorithm [3] and stored in blocks. SFC-Quad consumes the least space overheads. In contrast, ARM-SQ and HS-SKQ require additional space for materialized inverted lists, leading to slightly higher space overheads. The index size of HS-SKQ is significantly smaller than the other R-tree based indexes.

VI. CONCLUSION

Addressing the inefficiency of existing spatial keyword query algorithms, we introduce HQT-TI, a Hilbert curve based spatio-textual index, along with the corresponding spatial keyword query algorithm, HS-SKQ. The key improvement of HS-SKQ lies in its efficient spatial range query algorithm HQT-SQ. HQT-SQ employs a depth-first approach with limited breadth expansion strategy for spatial range queries, optimizing spatial cost to minimize overlap checks and significantly reducing the number of returned *IDRs*, *#AID*, and *#AIO*. Building on HQT-SQ, an efficient segment list intersection algorithm, SLI-KQ, further enhances the performance of HQT-SQ. Experimental results demonstrate that HS-SKQ outperforms existing algorithms significantly.

Although the query drill-down increases the spatial query cost, it significantly enhances the efficiency of keyword queries. Importantly, experiments show that optimal query efficiency can be achieved with relatively small *m*, thus limiting the introduced spatial query cost. Additionally, HS-SKQ works well in scenarios with both a few and many frequent keywords, addressing issues regarding skewed keyword distributions. Our future research will focus on enhancing algorithm performance by considering spatial skew distributions. Extending our algorithm to spatial-temporal keyword query is also an intriguing research direction.

ACKNOWLEDGMENT

Special thanks to professor Yufan Sheng, Xin Cao and Gao Cong for providing the WISK code.

REFERENCES

[1] L. Chen, G. Cong, and X. Cao, "An efficient query indexing mechanism for filtering geo-textual data," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2013, pp. 749–760.

[2] L. Chen, G. Cong, C. S. Jensen, and D. Wu, "Spatial keyword query processing: An experimental evaluation," *Proc. VLDB Endowment*, vol. 6, no. 3, pp. 217–228, 2013.

[3] M. Christoforaki, J. He, C. Dimopoulos, A. Markowetz, and T. Suel, "Text vs. space: Efficient Geo-search query processing," in *Proc. 20th ACM Int. Conf. Inf. Knowl. Manage.*, 2011, pp. 423–432.

[4] L. Jia, H. Tang, M. Li, B. Zhao, S. Wei, and H. Zhou, "An efficient association rule mining-based spatial keyword index," *Int. J. Data Warehousing Mining*, vol. 19, no. 2, 2023.

[5] J. Rao, J. Lin, and H. Samet, "Partitioning strategies for spatio-textual similarity join," in *Proc. 3rd ACM SIGSPATIAL Int. Workshop Analytics Big Geospatial Data*, 2014, pp. 40–49.

[6] J. Chen, L. Yu, and W. Wang, "Hilbert space filling curve based scan-order for point cloud attribute compression," *IEEE Trans. Image Process.*, vol. 31, pp. 4609–4621, 2022.

[7] Y. Lei, X. Tong, D. Wang, C. Qiu, H. Li, and Y. Zhang, "W-Hilbert: A W-shaped Hilbert curve and coding method for multiscale geospatial data index," *Int. J. Appl. Earth Observ. Geoinformation*, vol. 118, 2023, Art. no. 103298.

[8] L. Jia, Y. Fan, J. Ding, X. Li, and J. You, "3D Hilbert space filling curve encoding and decoding algorithms based on efficient prefix reduction," *J. Electron. Inf. Technol.*, vol. 46, no. 2, pp. 633–642, 2024.

[9] L. Jia et al., "Efficient group based Hilbert encoding and decoding algorithms," *Pattern Recognit.*, vol. 167, 2025, Art. no. 111718.

[10] B. Croft, D. Metzler, and T. Strohman, *Search Engines: Information Retrieval in Practice*, Boston, MA, USA: Addison-Wesley, 2009, pp. 1–552.

[11] Z. Li, K. C. Lee, B. Zheng, W.-C. Lee, D. Lee, and X. Wang, "IR-tree: An efficient index for geographic document search," *IEEE Trans. Knowl. Data Eng.*, vol. 23, no. 4, pp. 585–599, Apr. 2011.

[12] D. Wu, M. L. Yiu, G. Cong, and C. S. Jensen, "Joint top-K spatial keyword query processing," *IEEE Trans. Knowl. Data Eng.*, vol. 24, no. 10, pp. 1889–1903, Oct. 2012.

[13] Y. Zhou, X. Xie, C. Wang, Y. Gong, and W.-Y. Ma, "Hybrid index structures for location-based web search," in *Proc. 14th ACM Int. Conf. Inf. Knowl. Manage.*, 2005, pp. 155–162.

[14] D. Wu, G. Cong, and C. S. Jensen, "A framework for efficient spatial web object retrieval," *VLDB J.*, vol. 21, pp. 797–822, 2012.

[15] C. Zhang, Y. Zhang, W. Zhang, and X. Lin, "Inverted linear quadtree: Efficient top k spatial keyword search," *IEEE Trans. Knowl. Data Eng.*, vol. 28, no. 7, pp. 1706–1721, Jul. 2016.

[16] C. E. Sanjuan-Contreras, G. G. Retamal, M. A. Martínez-Prieto, and D. Seco, "cBIK: A space-efficient data structure for spatial keyword queries," *IEEE Access*, vol. 8, pp. 98827–98846, 2020.

[17] Z. Chen, L. Chen, G. Cong, and C. S. Jensen, "Location-and keyword-based querying of geo-textual data: A survey," *VLDB J.*, vol. 30, no. 4, pp. 603–640, 2021.

[18] L. Chen, S. Shang, C. Yang, and J. Li, "Spatial keyword search: A survey," *GeoInformatica*, vol. 24, pp. 85–106, 2020.

[19] P. Tampakis, D. Spyrellis, C. Doulkeridis, N. Pelekis, C. Kalyvas, and A. Vlachou, "A novel indexing method for spatial-keyword range queries," in *Proc. 17th Int. Symp. Spatial Temporal Databases*, 2021, pp. 54–63.

[20] A. Karabinos, P. Tampakis, C. Doulkeridis, and A. Vlachou, "Processing of spatial-keyword range queries in apache spark," in *Proc. 11th ACM SIGSPATIAL Int. Workshop Analytics Big Geospatial Data*, 2023, pp. 23–31.

[21] X. Ding, Y. Zheng, Z. Wang, K. R. Choo, and H. Jin, "A learned spatial textual index for efficient keyword queries," *J. Intell. Inf. Syst.*, vol. 60, no. 3, pp. 803–827, 2023.

[22] Y. Sheng et al., "WISK: A workload-aware learned index for spatial keyword queries," *Proc. ACM Manag. Data*, vol. 1, no. 2, pp. 187:1–187:27, 2023.

[23] X. Ge, J. Yu, and F. Kong, "Privacy-preserving Boolean range query with verifiability and forward security over spatio-textual data," *Inf. Sci.*, vol. 677, 2024, Art. no. 120929.

[24] Q. Tong, X. Li, Y. Miao, Y. Wang, X. Liu, and R. H. Deng, "Beyond result verification: Efficient privacy-preserving spatial keyword query with suppressed leakage," *IEEE Trans. Inf. Forensics Secur.*, vol. 19, pp. 2746–2760, 2024.

[25] Z. Ye, X. Wang, Z. Liu, D. Zhu, and J. Ma, "Obir-tree: An efficient oblivious index for spatial keyword queries on secure enclaves," *Proc. ACM Manag. Data*, vol. 3, no. 1, pp. 58:1–58:24, 2025.

[26] M. Orabi, Z. A. Aghbari, I. Kamel, and D. Mouheb, "Keeping an eye on moving objects: Processing continuous spatial-keyword range queries," *GeoInformatica*, vol. 28, no. 1, pp. 117–143, 2024.

- [27] M. Orabi, Z. Al Aghbari, and I. Kamel, "SkyEye: Continuous processing of moving spatial-keyword queries over moving objects," *GeoInformatica*, vol. 28, pp. 559–603, 2024.
- [28] C. Luo, Q. Liu, Y. Gao, L. Chen, Z. Wei, and C. Ge, "TASK: An efficient framework for instant error-tolerant spatial keyword queries on road networks," *Proc. VLDB Endow.*, vol. 16, no. 10, pp. 2418–2430, 2023.
- [29] P. Ahmed, A. Eldawy, V. Hristidis, and V. J. Tsotras, "Reverse spatial top-k keyword queries," *VLDB J.*, vol. 32, no. 3, pp. 501–524, 2023.
- [30] Y. Arseneau, S. Gautam, B. Nickerson, and S. Ray, "Stilt: Unifying spatial, temporal and textual search using a generalized multi-dimensional index," in *Proc. 32nd Int. Conf. Sci. Stat. Database Manage.*, 2020, pp. 1–12.
- [31] J. Li, J. Ma, Y. Miao, F. Yang, X. Liu, and K.-K. R. Choo, "Secure semantic-aware search over dynamic spatial data in VANETs," *IEEE Trans. Veh. Technol.*, vol. 70, no. 9, pp. 8912–8925, Sep. 2021.
- [32] Z. Zhang, J. Qi, X. Cao, and C. S. Jensen, "Semask: Answering semantics-aware spatial keyword queries with large language models," in *Proc. 28th Int. Conf. Extending Database Technol.*, Barcelona, Spain, 2025, pp. 1102–1105.
- [33] S. Liu, G. Cong, K. Feng, W. Gu, and F. Zhang, "Effectiveness perspectives and a deep relevance model for spatial keyword queries," *Proc. ACM Manag. Data*, vol. 1, no. 1, pp. 11:1–11:25, 2023.
- [34] L. Jia, D. Li, H. Zhou, and F. Xia, "Efficient list intersection algorithm for short documents by document reordering," *Mathematics*, vol. 12, no. 9, 2024, Art. no. 1328.



Suprio Ray received the PhD degree from the University of Toronto, in 2015. He is currently an associate professor with the University of New Brunswick. He has served on the Program Committees of SIGMOD, ICDE, EDBT, SSTD, MDM. His research interests include spatio-temporal data processing, Big Data systems, query processing on modern hardware, scalable data science, and data privacy and security.



Jiaman Ding received the master's degree from the Kunming University of Science and Technology, in 2005. He is currently a professor in the Kunming University of Science and Technology. His research interests include data mining, and Big Data processing.



Lianyin Jia received the PhD degree from the South China University of Technology, in 2012. He is currently a professor in Kunming University of Science and Technology. His research interests include databases, data mining, information retrieval, and parallel computing.



Xiaodong Fu received the PhD degree from the Kunming University of Science and Technology, in 2008. He is currently a professor in the Kunming University of Science and Technology. His research interests include services computing, federated learning, and intelligent decision making. He has served as a PC member for the IEEE ICWS and ICSOC.



Yongwang Miao is a postgraduate student with Kunming University of Science and Technology. His research interests mainly include database systems, data mining, and spatial keyword queries.



Xiuxing Li received the PhD degree from the Department of Computer Science and Technology, Tsinghua University, in 2022. He is currently an assistant researcher in the Institute of Computing Technology, Chinese Academy of Sciences, Beijing, China. His research interests mainly include data mining and knowledge graph. He is serving or has served as a PC member for some leading international conferences, such as SIGKDD and ICDE.